

# Memoria Transaccional por Software

## Software Transactional Memories (STM)

# Idea Central

- Reemplazar locks por bloques atómicos: `atomic {bloque}`
- **Garantiza:**
  - **Atomicidad:** Los efectos se hacen visibles todos juntos (los estados intermedios no son visibles a otros threads).
  - **Aislamiento (Isolation):** La ejecución de un bloque no se ve afectada por otros threads.
- Funciona de manera análoga a las transacciones **ACID**.

# Ventajas de STM

- **No introduce deadlocks** (ya que no se manejan locks explícitos).
- Son **composicionales**.
- La **gestión de errores es simple** (basta con levantar una excepción en el código).

# Motivación

```
class Cuenta {  
    private double saldo = 0;  
  
    synchronized void depositar(double monto) {  
        //pre: monto > 0  
        saldo += monto;  
    }  
    synchronized void extraer(double monto) {  
        //pre: monto > 0  
        if (saldo < monto)  
            throw new SaldoInsuficiente();  
        saldo -= monto;  
    }  
  
    synchronized void transferir(Cuenta cuenta,  
        double monto) {  
        cuenta.extraer(monto);  
        depositar(monto);  
    }  
}
```

# Motivación

```
class Cuenta {  
    private double saldo = 0;  
  
    synchronized void depositar(double monto) {  
        //pre: monto > 0  
        saldo += monto;  
    }  
    synchronized void extraer(double monto) {  
        //pre: monto > 0  
        if (saldo < monto)  
            throw new SaldoInsuficiente();  
        saldo -= monto;  
    }  
  
    synchronized void transferir(Cuenta cuenta,  
        double monto) {  
        cuenta.extraer(monto);  
        depositar(monto);  
    }  
}
```

## Posibilidad de Deadlock

Dos transferencias concurrentes en sentido opuesto entre las mismas cuentas pueden producir un deadlock.

## Problema en el uso de Locks

- El uso tradicional de locks y variables de condición **no es composicional**.
- Esto afecta y perjudica a la **programación modular**.

# Idea Principal e Intuición de STM

## Idea

Introducir un constructor atómico para delimitar el bloque de código:

```
atomic { bloque de código }
```

```
void transferir(Cuenta cuenta,  
    double monto) {  
    atomic {  
        cuenta.extraer(monto);  
        depositar(monto);  
    }  
}
```

# Idea Principal e Intuición de STM

## Idea

Introducir un constructor atómico para delimitar el bloque de código:

```
atomic { bloque de código }
```

```
void transferir(Cuenta cuenta,  
    double monto) {  
    atomic {  
        cuenta.extraer(monto);  
        depositar(monto);  
    }  
}
```

## Intuición sobre la ejecución:

- Ejecutar el bloque **sin tomar locks**.
- Registrar cada lectura y escritura en un **log local** de cada thread.
- Escribir únicamente en el log local (no en la memoria global).
- **Al finalizar, se valida el log:**
  - **Si es válido:** Se actualiza atómicamente la memoria principal.
  - **Si no es válido:** Se descartan los cambios y se recomienza desde el principio.

## Dificultad para la implementación

- **Problema:** Loggear los efectos secundarios.
- En ciertos lenguajes funcionales se distinguen de forma estricta las expresiones puras de aquellas que tienen efectos.
- Esto representa una excelente **oportunidad para las mónadas**.

- **Funciones puras:** Solo calculan y retornan un resultado basado estrictamente en sus parámetros.
  - una función aplicada a los mismos argumentos, retorna el mismo resultado.
- **Ausencia de efectos secundarios (Side Effects):** Evaluación de las expresiones no pueden alterar el entorno. .

## Transparencia referencial:

El valor de una expresión depende sólo del valor de sus subexpresiones (no depende de la historia de la ejecución ni del orden de evaluación)

# ¿Cómo hace Haskell para I/O?

- Operaciones de I/O no son puras (escriben/leen archivo, pantallas, teclado)
- **La solución de Haskell: Expresiones separado por el sistema de tipos.**

## El Mundo Puro

- Libre de efectos secundarios.
- Permite razonar con transparencia referencial.

## El Mundo Impuro

- Se encarga del “trabajo sucio”.
- Interactúa con el teclado, la pantalla y el exterior.

# ¿Cómo hace Haskell para I/O?

- Operaciones de I/O no son puras (escriben/leen archivo, pantallas, teclado)
- **La solución de Haskell: Expresiones separadas por el sistema de tipos.**

## El Mundo Puro

- Libre de efectos secundarios.
- Permite razonar con transparencia referencial.

## El Mundo Impuro

- Se encarga del “trabajo sucio”.
- Interactúa con el teclado, la pantalla y el exterior.

## Tipos Monádicos (Mónadas)

El tipo de las expresiones impuras.

## Tipos monádicos: Intuición

### ¿Qué es un Tipo Monádico?

Un tipo común dice **los posibles valores** (un entero, un texto, un booleano) de una expresión tienes. Un **tipo monádico** dice los posibles valores **Y ADEMÁS** en qué contexto existe ese valor.

# Tipos monádicos: Intuición

## ¿Qué es un Tipo Monádico?

Un tipo común dice **los posibles valores** (un entero, un texto, un booleano) de una expresión tienes. Un **tipo monádico** dice los posibles valores **Y ADEMÁS** en qué contexto existe ese valor.

## Ejemplos

- `Int`: Es un entero puro ordinario.
- `Maybe Int`: Es un tipo monádico. Es un entero que habita en el contexto de la **opcionalidad** (el cómputo puede fallar).
- `[Int]`: Es un tipo monádico. Es un entero que habita en el contexto de múltiples valores posibles (**no-determinismo**).
- `IO Int`: Es un tipo monádico. Es un entero que habita en el contexto de una **computación imperativa**.

# Constructores de Tipos

## Tipos Concretos

- Representan valores directos en memoria.
- Tipos como `Int`, `Char` o `Bool`.
- Su clasificación o *Kind* es `*`.

# Constructores de Tipos

## Tipos Concretos

- Representan valores directos en memoria.
- Tipos como `Int`, `Char` o `Bool`.
- Su clasificación o *Kind* es `*`.

## Constructores de Tipos (Fábricas)

- Plantillas de tipos. Necesitan aplicarse a un tipo para crear un tipo concreto.
- Constructores como `Maybe`, `[]` (Lista) o `(Bool,)`.
- Su *Kind* es `* -> *`.

## Clase de Tipos (Typeclass)

Es una interfaz que define un **conjunto de operaciones**

- Permite lograr **polimorfismo ad-hoc** (sobrecarga de funciones).
- Un tipo de dato concreto puede elegir “implementar” esta interfaz creando una **instancia**.

# Abstracción de Comportamiento: Clases de Tipos

## Clase de Tipos (Typeclass)

Es una interfaz que define un **conjunto de operaciones**

- Permite lograr **polimorfismo ad-hoc** (sobrecarga de funciones).
- Un tipo de dato concreto puede elegir “implementar” esta interfaz creando una **instancia**.

## Ejemplo: La clase Eq (Igualdad)

```
-- Definicion de la interfaz
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool
    {-# MINIMAL (==) | (/=) #-}

-- Instancia para un tipo propio
data Estado = Activo | Inactivo

instance Eq Estado where
    Activo    == Activo    = True
    Inactivo  == Inactivo  = True
    _         == _         = False
```

# La Clase Functor: Contenedores mapeables

```
type Functor :: (* -> *) -> Constraint
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  ...
{-# MINIMAL fmap #-}
```

- Es una clase de tipos que representa a estructuras o contenedores que pueden ser **mapeados**.
- `fmap`
  - Toma una función pura `(a -> b)`.
  - Desempaque un valor `(f a)`
  - aplica la función al valor y devuelve el resultado empaquetado con tipo `(f b)`.

## Instancias comunes:

```
-- Maybe
fmap (+1) (Just 5)  -- Just 6
fmap (+1) Nothing   -- Nothing
```

# La Clase Functor: Contenedores mapeables

```
type Functor :: (* -> *) -> Constraint
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  ...
{-# MINIMAL fmap #-}
```

- Es una clase de tipos que representa a estructuras o contenedores que pueden ser **mapeados**.
- `fmap`
  - Toma una función pura `(a -> b)`.
  - Desempaque un valor `(f a)`
  - aplica la función al valor y devuelve el resultado empaquetado con tipo `(f b)`.

## Instancias comunes:

```
-- Maybe
fmap (+1) (Just 5)  -- Just 6
fmap (+1) Nothing   -- Nothing

-- Listas (fmap es map)
fmap (+1) [1, 2, 3] -- [2, 3, 4]
```

# La Clase Functor: Contenedores mapeables

```
type Functor :: (* -> *) -> Constraint
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  ...
{-# MINIMAL fmap #-}
```

- Es una clase de tipos que representa a estructuras o contenedores que pueden ser **mapeados**.
- `fmap`
  - Toma una función pura (`a -> b`).
  - Desempaque un valor (`f a`)
  - aplica la función al valor y devuelve el resultado empaquetado con tipo (`f b`).

## Instancias comunes:

```
-- Maybe
fmap (+1) (Just 5)  -- Just 6
fmap (+1) Nothing   -- Nothing

-- Listas (fmap es map)
fmap (+1) [1, 2, 3] -- [2, 3, 4]

-- Pairs (es aplicar a snd)
fmap (+1) (True, 2) -- (True, 3)
```

# La Clase Functor: Contenedores mapeables

```
type Functor :: (* -> *) -> Constraint
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  ...
{-# MINIMAL fmap #-}
```

- Es una clase de tipos que representa a estructuras o contenedores que pueden ser **mapeados**.
- `fmap`
  - Toma una función pura `(a -> b)`.
  - Desempaque un valor `(f a)`
  - aplica la función al valor y devuelve el resultado empaquetado con tipo `(f b)`.

## Instancias comunes:

```
-- Maybe
fmap (+1) (Just 5)  -- Just 6
fmap (+1) Nothing   -- Nothing

-- Listas (fmap es map)
fmap (+1) [1, 2, 3] -- [2, 3, 4]

-- Pairs (es aplicar a snd)
fmap (+1) (True, 2) -- (True, 3)

-- Flecha (es composición)
fmap 3) id 10 - 30
```

# Funtores aplicativos

```
type Applicative :: (* -> *) ->
  Constraint
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
  ...
```

- Todo tipo Applicative debe ser un Functor.
- Es un functor que permite aplicar funciones que **ya se encuentran dentro de un contexto**.
- `pure`: Toma un valor y lo introduce en un contexto mínimo (por defecto).
- `<*>` : Aplica una función en un contexto a un valor en un contexto del mismo tipo.

```
-- Maybe

pure 5 :: Maybe Int -- Just 5

Just (+3) <*> Just 2 -- Just 5
Nothing <*> Just 2 -- Nothing
Just (+3) <*> Nothing -- Nothing
```

# Functores aplicativos

```
type Applicative :: (* -> *) ->
  Constraint
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
  ...
```

- Todo tipo Applicative debe ser un Functor.
- Es un functor que permite aplicar funciones que **ya se encuentran dentro de un contexto**.
- **pure**: Toma un valor y lo introduce en un contexto mínimo (por defecto).
- **<\*>** : Aplica una función en un contexto a un valor en un contexto del mismo tipo.

```
-- Maybe
```

```
pure 5 :: Maybe Int -- Just 5
```

```
Just (+3) <*> Just 2 -- Just 5
Nothing <*> Just 2 -- Nothing
Just (+3) <*> Nothing -- Nothing
```

```
-- Lists
```

```
pure 5 :: [Int] -- [5]
```

```
[] <*> [10, 20] -- []
[id, (+1), (*2)] <*> [] --- []
[id, (+1), (*2)] <*> [10,20] --
  [10,20,11,21,20,40]
```

# La Definición Formal: Clase Monad

## La firma en el preludio:

```
class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
  ...
```

- Actúa como una “estrategia de combinación” para encadenar cálculos con un contexto.

# La Definición Formal: Clase Monad

## La firma en el preludio:

```
class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
  ...
```

- Actúa como una “estrategia de combinación” para encadenar cálculos con un contexto.

- `return` (Inyección): Toma un valor puro y lo empaqueta en la mónada `m`.
- `>>=` (El operador *Bind*): Toma un valor empaquetado (`m a`), extrae el valor interno y se lo pasa a una función que genera el siguiente valor empaquetado (`m b`).

# El Problema: Operar a Mano con Maybe

## Evaluación manual con case:

```
-- Evaluar 'odd' sobre un 'Maybe Int'
evaluarOdd :: Maybe Int -> Maybe Bool
evaluarOdd mi =
  case mi of
    Nothing -> Nothing
    Just x   -> Just (odd x)
```

### No escala

Si tuviéramos un pipeline de 3 operaciones consecutivas que devuelven `Maybe`, ¡terminaríamos con una pirámide de 3 niveles de `case` anidados!

- Obliga al programador a escribir explícitamente el manejo del caso de error (`Nothing`) en cada paso.
- Oscurece la lógica con código de control repetitivo.
- Rompe la fluidez de la composición de funciones puras.

# Manejo de Opcionalidad: La Mónada Maybe

## La instancia monádica:

```
instance Monad Maybe where
    return x      = Just x
    Nothing >>= _ = Nothing
    Just x  >>= f = f x
```

- Encapsula cálculos que pueden **fallar** o no devolver un valor.
- El operador `>>=` actúa como un fusible: si encuentra un `Nothing`, detiene la cadena inmediatamente.

# Manejo de Opcionalidad: La Mónada Maybe

## La instancia monádica:

```
instance Monad Maybe where
    return x      = Just x
    Nothing >>= _ = Nothing
    Just x  >>= f = f x
```

- Encapsula cálculos que pueden **fallar** o no devolver un valor.
- El operador >>= actúa como un fusible: si encuentra un **Nothing**, detiene la cadena inmediatamente.

## Usando el operador Bind (»=)

```
evaluarOdd :: Maybe Int -> Maybe
             Bool
evaluarOdd mi = mi >>= (\x ->
    return (odd x))
```

## Anidamiento con `>>=`:

```
foo :: Maybe String
foo = Just 3      >>= (\ x ->
    Just " ! " >>= (\ y ->
        Just (show x ++ y)))
```

### Scope

Al anidar los lambdas, la variable `x` definida en el bloque más externo sigue estando en el *scope* de la expresión final.

## ¿Cómo fluyen los datos?

- El operador `>>=` extrae el 3 y el string `"!"` de sus respectivos contenedores `Maybe`.
- Durante la ejecución, los identificadores `x` e `y` actúan como variables puras ordinarias.
- El código se vuelve difícil de leer debido a la acumulación de paréntesis y estructuras tipadas.

# La Notación do

```
foo :: Maybe String
foo = do
  x <- Just 3
  y <- Just " ! "
  Just (show x ++ y)
```

- Las flechas <- se traducen en el operador >=>
- Cada nueva línea abre una lambda implícita hacia abajo.

## Conservación del Scope

La notación `do` mantiene el scope de las variables: `x` e `y` están definidas en las líneas siguientes.

# Mónada IO (Computación imperativa)

## Tipo monádico IO a

- Representa una operación que, al ejecutarse, realiza una tarea con **efectos secundarios** (escribir en pantalla, leer del teclado, etc.).
- Produce o “devuelve” un valor de tipo a al finalizar.

```
putStrLn :: String -> IO ()
```

```
putStrLn "hello" :: IO ()
```

## IO()

Una **acción de I/O** con resultado `unit` (tipo `()`).

# El Punto de Entrada: la función `main`

## Función `main`

- **Punto de entrada** de todo programa ejecutable en Haskell.
- Tiene el tipo `main :: IO a`.
- Le indica al compilador y al sistema operativo dónde comienza la ejecución secuencial de los efectos secundarios.

```
-- Archivo: Main.hs  
main :: IO ()  
main = putStrLn "hello, world"
```

Al compilar y ejecutar el programa, la acción imperativa se ejecuta, muestra la cadena en la terminal y finaliza retornando `()`.

# Composición de Acciones: La Notación do

## Sintaxis do

- Permite **pegar/secuenciar** múltiples acciones individuales en una sola gran acción.
- Todas las líneas del bloque deben ser del tipo IO.
- No se pueden incluir expresiones puras de forma directa.
- Se lee de forma similar a un **programa imperativo** tradicional (pero no es lo mismo)

```
main :: IO ()  
main = do  
    putStrLn "Hello,"  
    putStrLn "world!"
```

## El Tipo de la Composición

La acción resultante adopta el tipo de la **última acción** de I/O del bloque.

# Expresión Pura vs. Acción con Efectos

## Expresión Pura

```
"Hola" :: String
```

- Un dato en memoria.
- Evaluación libre de efectos secundarios.
- No “hace” nada ni modifica el contexto.

## Acción con Efectos

```
putStrLn "Hola" :: IO ()
```

- Comando imperativo que realiza I/O.
- Al ejecutarse, interactúa con el entorno.
- Retorna un valor de tipo `unit ()`.

## Sistema de tipos de Haskell

Garantiza que no se puede usar una acción IO dentro de una función pura.  
**¡Los mundos se mantienen separados!**

# Lectura y Desempaquetado: `getLine`

```
getLine :: IO String
```

## Empaquetado:

- Una acción `IO` a hace algo y produce un valor de tipo `a`.
- La **única forma** de acceder al valor es desempaquetarlo.
- **Restricción:** Solo se puede extraer datos de una acción `IO` dentro de otra acción `IO`.

## Operador bind `<-`:

- `name <- getLine` : Ejecuta la acción `getLine`, desempaqueta el resultado y lo asocia al identificador `name`.
- Como la acción `name <- getLine` es de tipo `IO String`, la variable `name` tiene tipo `String` (puro).

## `getLine` es Impura

Si se ejecuta dos veces puede producir valores distintos.

# Ejemplos Prácticos de Interacción

```
main :: IO ()
main = do
    putStrLn "Hola, cómo te llamas?"
    nombre <- getLine
    putStrLn ("Hola " ++ nombre ++ "!")
```

Cada paso se ejecuta de forma secuencial. El valor extraído con <- se concatena en el último putStrLn.

```
-- procesar :: String -> String

main :: IO ()
main = do
    putStrLn "Hola, cómo te llamas?"
    nombre <- getLine
    putStrLn ("Hola: " ++ procesar
              nombre)
```

Podemos aplicar funciones puras (como procesar) al valor desempaquetado.

# Errores Comunes: Mezclar Tipos Incompatibles

## El contraejemplo incorrecto:

```
-- CODIGO ERRONEO: No compila  
nameTag = "Hola " ++ getLine
```

# Errores Comunes: Mezclar Tipos Incompatibles

## El contraejemplo incorrecto:

```
-- CODIGO ERRONEO: No compila  
nameTag = "Hola " ++ getLine
```

### Error de Tipos

Haskell rechazará esto porque intentamos concatenar un `String` con una acción `IO String`.

## ¿Por qué falla?

- El operador `++` es un operador binario sobre `String`.
- `getLine` **no es** una `String`, es una acción para obtener una `String`.
- Para usar el valor debemos estar dentro de un bloque `do` y extraerlo usando `<-`.

# Valores de Retorno y Enlaces Simbólicos

```
main :: IO ()
main = do
  foo <- putStrLn "Hello, name?"
  name <- getLine
  putStrLn ("Hey " ++ name)
```

- Cada acción **IO** produce un resultado al ejecutarse.
- **putStrLn** retorna ().
- Hacer un bind funciona, pero es irrelevante

## Regla

La última acción de un bloque **do** **no puede** ligarse a un nombre usando **<-**.

- El bloque **do** tiene como valor el valor de la **última acción**.

# Uso de let dentro de Bloques de I/O

```
import Data.Char

main :: IO ()
main = do
    putStrLn "Nombre?"
    nombre <- getLine
    putStrLn "Apellido?"
    apellido <- getLine
    let nombreM = map toUpper nombre
        apellidoM = map toUpper apellido
    putStrLn ("Hola " ++ nombreM ++ " "
              ++ apellidoM ++ "!")
```

## Let vs bind:

- <- ejecuta una acción IO y desempaqueta el resultado.
- **let** asigna un alias a una **expresión pura**. No requiere `in` al final cuando está dentro de un `do`.

## La Indentación Importa

Expresiones dentro de `let` y `do` deben estar indentados.

# Estudio de Caso: Invertir Palabras

```
main :: IO ()
main = do
  line <- getLine
  if null line
    then return ()
    else do
      putStrLn (reverseWords line)
      main

reverseWords :: String -> String
reverseWords = unwords . map
               reverse . words
```

- **Condicional:** En Haskell, todo `if` requiere obligatoriamente su `else`.
- **Bloque `else`:** Agrupa dos acciones (`putStrLn` y la llamada recursiva a `main`) encapsulándolas en un sub-bloque `do`.
- **Bloque `then`:** `return` empaqueta un valor puro en una acción `IO`:  
`return :: a -> IO a`

# ¿Qué hace return?

**return** es el dual de **<-**:

- **return** NO finaliza la ejecución del bloque ni rompe el flujo del programa.
- Su única función es tomar un valor puro y **envolverlo en una acción** IO (introducirlo en una caja).
- **return** () crea una acción ficticia que no altera el mundo real y solo entrega el valor vacío.

**Flujo continuo sin interrupciones:**

```
main :: IO ()
main = do
    return ()
    return "HAHAHA"
    line <- getLine
    putStrLn line
```

## Uso Técnico de return

Se emplea principalmente al final de un bloque do cuando queremos forzar que la macro-acción completa entregue un resultado específico en lugar del valor de su última instrucción.

# Entrada y Salida: Primitivas de Datos

## Primitivas esenciales:

- `putStr :: String -> IO ()`  
Imprime texto sin salto de línea.
- `putChar :: Char -> IO ()`  
Imprime un único carácter.
- `getLine :: IO String`  
Lee una línea desde la terminal.
- `print :: Show a => a -> IO ()`  
Aplica `show` e imprime con salto de línea.

## Ejemplo práctico:

```
main :: IO ()
main = do
    putStr "Nombre: "
    line <- getLine
    putStr "Hola, "
    putStrLn line
    putStr "Inicial: "
    putChar 'X'
```

# Estructuras de Control y Combinadores

## Combinadores lógicos:

- `when :: Bool -> IO () -> IO ()`  
Ejecuta la acción condicional únicamente si el booleano es verdadero.
- `forever :: IO a -> IO b`  
Ejecuta una acción de forma infinitamente cíclica.
- `forM :: [a] -> (a -> IO b) -> IO [b]`  
Itera sobre una lista mapeando acciones (estilo *foreach*).

## Ejemplo con iteradores:

```
import Control.Monad (when,
                      forever)
import Data.Traversable (forM)

main :: IO ()
main = do
    let valores = [1, 2, 3, 4]
    forM valores (\v -> do
        when (odd v) (print v))
    return ()
```

*Nota:* `mapM` es idéntico a `forM` pero toma la lista como su segundo argumento.

# Estructuras de Control y Combinadores

## Combinadores lógicos:

- `when :: Bool -> IO () -> IO ()`  
Ejecuta la acción condicional únicamente si el booleano es verdadero.
- `forever :: IO a -> IO b`  
Ejecuta una acción de forma infinitamente cíclica.
- `forM :: [a] -> (a -> IO b) -> IO [b]`  
Itera sobre una lista mapeando acciones (estilo *foreach*).

## Ejemplo con iteradores:

```
import Control.Monad (when)
import Data.Traversable (forM)

main :: IO ()
main = do
    let nombres = ["Ana", "Bo", "Carlos"]
    forM nombres (\n -> do
        when (length n > 2) (
            putStrLn n))
    return ()
```

# Estado Mutable en un Mundo Puro: IORef

## ¿Qué es una IORef?

- Es el mecanismo nativo más simple de Haskell para simular variables mutables tradicionales.
- Funciona de forma análoga a un puntero o una celda de memoria.
- Se importa desde el módulo `Data.IORef`.

```
newIORef    :: a -> IO (IORef a)
readIORef   :: IORef a -> IO a
writeIORef  :: IORef a -> a -> IO ()
```

- Como modificar la memoria es un efecto, todas sus expresiones tiene **tipo monádico IO**.
- **Inicialización:** No pueden estar vacías; se les da un valor inicial al crearse.

# Contador imperativo

```
import Data.IORef

incRef :: IORef Int -> IO ()
incRef var = do
    val <- readIORef var
    writeIORef var (val + 1)

main :: IO ()
main = do
    var <- newIORef 42
    incRef var
    val <- readIORef var
    print val -- Imprime 43
```

# Contador imperativo

```
import Data.IORef

incRef :: IORef Int -> IO ()
incRef var = do
    val <- readIORef var
    writeIORef var (val + 1)

main :: IO ()
main = do
    var <- newIORef 42
    incRef var
    val <- readIORef var
    print val -- Imprime 43
```

## No son thread-safe:

- Si múltiples hilos intentan ejecutar `incRef` en paralelo sobre la misma referencia, ocurrirá una **condición de carrera**.
- La lectura y la escritura se separan en dos pasos distintos, rompiendo la atomicidad.

# Threads en Haskell: forkIO

- Permiten evaluar una expresión en hilo de ejecución concurrente.
- Es manejado por el *Runtime* de GHC.
- Se importa el módulo `Control.Concurrent`.

```
forkIO :: IO () -> IO ThreadId
```

```
import Control.Concurrent

main :: IO ()
main = do
    forkIO (putStr "Hello")
    putStr " world\n"
```

## Asincronía y el Hilo Principal

- `forkIO` dispara el hilo y retorna inmediatamente sin esperar a que la acción termine.
- Si el hilo principal (`main`) finaliza, todos los hijos mueren instantáneamente.

# Demostración de Condición de Carrera con IORef

```
import Control.Concurrent (forkIO, threadDelay)
import Control.Monad (replicateM_)
import Data.IORef

worker :: IORef Int -> IO ()
worker ref = replicateM_ 1000000 (do
    v <- readIORef ref
    writeIORef ref (v + 1))

main :: IO ()
main = do
    ref <- newIORef 0
    forkIO (worker ref) -- Hilo hijo 1
    forkIO (worker ref) -- Hilo hijo 2
    threadDelay 50000 -- Feo (ver Async)
    final <- readIORef ref
    putStr "Resultado Final: "
    print final
```

## Resultado:

- **Esperado:** 2000000.
- **Real:** Un número menor y diferente en cada ejecución.
- El incremento no es atómico.

# Sincronización Controlada: La Librería async

## Abstracción de alto nivel:

- Representa los hilos concurrentes como promesas.
- Permite capturar resultados o sincronizar la finalización de forma nativa.
- Se importa desde el módulo `Control.Concurrent.Async`.

```
async :: IO a -> IO (Async a)
wait  :: Async a -> IO a
```

```
import Control.Concurrent.Async

main :: IO ()
main = do
    -- crea hilo y guarda promesa
    hilo <- async (putStr "Hello")
    putStr " world\n"
    -- Forzar la espera
    wait hilo
```

## Control del Ciclo de Vida

A diferencia de `forkIO`, la función `wait` bloquea al hilo principal de forma limpia hasta que la acción concurrente termine.

# Demostración de Condición de Carrera con IORef

```
import Control.Concurrent.Async (async, wait)
import Control.Monad (replicateM_)
import Data.IORef

worker :: IORef Int -> IO ()
worker ref = do
    replicateM_ 1000000 ( do
        v <- readIORef ref
        writeIORef ref ( v + 1) )
    putStrLn "Termina worker"

main :: IO ()
main = do
    ref <- newIORef 0
    t1 <- async (worker ref) -- Hilo hijo 1
    t2 <- async (worker ref) -- Hilo hijo 2
    wait t1
    wait t2
    final <- readIORef ref
    putStr "Resultado Final : "
    print final
```

## Resultado:

- **Esperado:** 2000000.
- **Real:** Un número menor y diferente en cada ejecución.
- El incremento no es atómico.

# STM y el Tipo IO

Quisiéramos poder definir una función:

```
atomic :: IO a -> IO a
```

Para usarla de la siguiente forma:

```
main = do
  forkIO (atomic (putStr "Hello"))
  atomic (putStr " world\n")
```

# STM y el Tipo IO

Quisiéramos poder definir una función:

```
atomic :: IO a -> IO a
```

Para usarla de la siguiente forma:

```
main = do
  forkIO (atomic (putStr "Hello"))
  atomic (putStr " world\n")
```

Sin embargo, consideremos este escenario:

```
main = do
  r <- newIORef 0
  forkIO (atomic (incRef r))
  (incRef r)
```

## El Sistema de Tipos

El sistema de tipos no puede garantizar que las referencias se utilicen exclusivamente dentro de transacciones. Cualquier hilo podría modificar `r` por fuera usando `writeIORef`.

# Tipos para Atomicidad (STM)

```
data STM a    -- abstracto
data TVar a   -- abstracto

instance Monad STM

newTVar      :: a -> STM (TVar a)
readTVar     :: TVar a -> STM a
writeTVar    :: TVar a -> a -> STM ()

atomically  :: STM a -> IO a
```

- Se introduce el tipo monádico STM y variables transaccionales independientes (TVar).
- Las variables TVar solo pueden modificarse dentro de transacciones.

# Tipos para Atomicidad (STM)

```
data STM a    -- abstracto
data TVar a   -- abstracto

instance Monad STM

newTVar      :: a -> STM (TVar a)
readTVar     :: TVar a -> STM a
writeTVar    :: TVar a -> a -> STM ()

atomically  :: STM a -> IO a
```

- Se introduce el tipo monádico STM y variables transaccionales independientes (TVar).
- Las variables TVar solo pueden modificarse dentro de transacciones.

## Garantía de tipos:

- Como readTVar y writeTVar devuelven acciones tipo STM, es imposible usarlas por fuera de una transacción.
- atomically actúa como el único puente al tipo monádico IO.

# Contador atómico STM

```
import Control.Concurrent
import Control.Concurrent.STM

-- Incremento Atómico
incT :: TVar Int -> IO ()
incT r = atomically (do
    v <- readTVar r
    writeTVar r (v + 1))

main :: IO ()
main = do
    r <- atomically (newTVar 0)
    forkIO (incT r)
    incT r
    val <- atomically (readTVar r)
    print val
```

- El bloque `do` dentro de `incT` tiene tipo STM a.
- La función `atomically` toma algo del tipo STM y lo ejecuta de forma atómica.

# Contador atómico STM

```
incT :: TVar Int -> IO ()
incT r = atomically (do
    v <- readTVar r
    writeTVar r (v + 1))

worker :: TVar Int -> IO ()
worker ref = do
    replicateM_ 1000000 (incT ref)

main :: IO ()
main = do
    ref <- atomically (newTVar 0)
    t1 <- async (worker ref) -- Hilo hijo 1
    t2 <- async (worker ref) -- Hilo hijo 2
    wait t1
    wait t2
    final <- atomically (readTVar ref)
    putStr "Resultado Final: "
    print final
```

## Resultado:

- **Esperado:** 2.000.000.
- **Real:** Exactamente **2.000.000** de forma determinista.
- Cada llamada a incT se ejecuta de forma atómica.
- Si hay colisión, el motor transaccional detecta la invalidez del log local, descarta los cambios y reintenta la operación.

# Composicionalidad

```
-- (reutilizable)
incTransaccional :: TVar Int -> STM ()
incTransaccional r = do
    v <- readTVar r
    writeTVar r (v + 1)

-- Composicion de dos incrementos
incT2 :: TVar Int -> IO ()
incT2 r = atomically (do
    incTransaccional r
    incTransaccional r)
```

- **Atomicidad:** Las dos llamadas a `incTransaccional` en `incT2` se ejecutan dentro del mismo `atomically`.
- Todo el bloque se valida y se impacta en la memoria.
- Ningún hilo concurrentes puede ver un estado intermedio donde la variable se haya incrementado una vez.

# Incremento transaccional

```
import Control.Concurrent
import Control.Concurrent.STM

incT :: TVar Int -> STM ()
incT r = do
    v <- readTVar r
    writeTVar r (v + 1)

main :: IO ()
main = do
    r <- atomically (newTVar 0)
    forkIO (atomically (incT r))
    atomically (incT r)
    val <- atomically (readTVar r)
    print val
```

- La función `incT` define una secuencia de operaciones dentro de la mónada `STM`.
- No es atómica por sí misma; puede combinarse con otros cálculos transaccionales para formar bloques más complejos.
- Es la función `atomically` la que toma la receta de tipo `STM` y la ejecuta de forma efectivamente atómica y aislada frente a los demás hilos concurrentes.

# Cuenta bancaria como STM

## Diseño: Cuentas como Variables

- Cada cuenta se modela como una variable (TVar Double).
- Las funciones describen cómo mutar el estado, pero no aplican los cambios.

```
extraer :: TVar Double -> Double -> STM ()
```

```
extraer cta n = do  
    saldo <- readTVar cta  
    writeTVar cta (saldo - n)
```

```
depositar :: TVar Double -> Double -> STM ()
```

```
depositar cta n = do  
    saldo <- readTVar cta  
    writeTVar cta (saldo + n)
```

```
transferir :: TVar Double -> TVar Double -> Double -> STM ()
```

```
transferir de en n = do  
    extraer de n  
    depositar en n
```

# Haciendo una transferencia

```
main :: IO ()
main = do
  a <- atomically (newTVar 10)  -- crea cuenta a
  b <- atomically (newTVar 5)   -- crea cuenta b
  atomically (transferir a b 4) -- transfiere atómicamente
  sa <- atomically (readTVar a)
  putStrLn (show sa)
```

- **Imposibilidad de deadlock** STM maneja un log local. Si hay conflicto, la transacción simplemente se aborta y se reintenta.

# Control de Condiciones: El Operador `retry`

**Operador** `retry` :: STM a

- Al ejecutar `retry`, la transacción se aborta inmediatamente y se reintentará más tarde.
- Reintentar de forma inmediata sería ineficiente.
- La implementación extrae del log de lectura las dependencias y lo despierta cuando otra transacción concurrente modifique las dependencias.

# Control de Condiciones: El Operador `retry`

## Operador `retry` :: STM a

- Al ejecutar `retry`, la transacción se aborta inmediatamente y se reintenta más tarde.
- Reintentar de forma inmediata sería ineficiente.
- La implementación extrae del log de lectura las dependencias y lo despertada cuando otra transacción concurrente modifique las dependencias.

## Control de saldo suficiente:

```
extraerSi :: TVar Double -> Double ->
  STM ()
extraerSi cta n = do
  saldo <- readTVar cta
  if saldo < n
  then retry
  else writeTVar cta (saldo - n)
```

# La Función check

Como el patrón de verificar una condición booleana es muy común, se define check:

```
check :: Bool -> STM ()
check True  = return ()
check False = retry

extraerSi cta n = do
    saldo <- readTVar cta
    check (n <= saldo)
    writeTVar cta (saldo - n)
```

# Ejemplo de Sincronización con Retardo

```
depositoRetardo cta monto = do
  threadDelay 3000000
  putStrLn "Depositando!"
  atomically (do
    saldo <- readTVar cta
    writeTVar cta (saldo + monto))

mainRetardo = do
  cta <- atomically (newTVar 100)
  putStrLn "Preparando deposito..."
  forkIO (depositoRetardo cta 10)
  putStrLn "Tratando de retirar..."
  atomically (extraerSi cta 101)
  putStrLn "Retire!"
```

# Composición Alternativa con orElse

- `orElse a1 a2` intenta ejecutar primero la acción `a1`.
- Si `a1` invoca un `retry`, sus efectos locales se descartan e intenta inmediatamente con `a2`.
- Si `a2` también reintenta, la composición duerme al hilo hasta que cambie alguna de las dependencias de alguna de las ramas.

## Extracción alternativa:

```
extraerAlt :: TVar Double -> TVar  
            Double -> Double -> STM ()  
extraerAlt cta1 cta2 monto =  
    orElse (extraerSi cta1 monto)  
           (extraerSi cta2 monto)
```

# Composición Alternativa y Orquestación con orElse

```
extraerAlt :: TVar Double -> TVar Double -> Double -> STM ()
extraerAlt cta1 cta2 monto =
    orElse (extraerSi cta1 monto)
           (extraerSi cta2 monto)

main :: IO ()
main = do
    cta1 <- newTVarIO 100
    cta2 <- newTVarIO 200
    atomically (extraerAlt cta1 cta2 150)
    s1 <- readTVarIO cta1
    s2 <- readTVarIO cta2
    putStrLn ("Saldos. cta1: " ++ show s1 ++ " cta2: " ++ show s2)
```

# Garantía de Tipos en STM

```
-- Error de compilación por uso de conEfecto
conEfecto :: IO ()
conEfecto = putStr "Todo bien"

main :: IO ()
main = do
  x <- atomically (newTVar 2)
  y <- atomically (newTVar 1)
  atomically (do
    a <- readTVar x
    b <- readTVar y
    if a > b then conEfecto else return ())
```

## Tipado

- El sistema de tipos detecta que `conEfecto` es de tipo `IO ()`.
- No es imposible inyectar una acción con efectos en un bloque monádico STM a.
- La mónada STM no tiene efectos `IO`.

## Caso de Estudio: Un Buffer Transaccional

- Un buffer es una referencia a una lista.
- `put` agrega un elemento al final.
- `get` es bloqueante cuando el buffer es vacío. La función `retry` suspende al hilo consumidor hasta que un productor altere el estado con un `put`.

# Caso de Estudio: Un Buffer Transaccional

- Un buffer es una referencia a una lista.
- put agrega un elemento al final.
- get es bloqueante cuando el buffer es vacío. La función retry suspende al hilo consumidor hasta que un productor altere el estado con un put.

```
type Buf a = TVar [a]

newBuf = newTVar []

put :: Buf a -> a -> STM ()
put b x = do
  xs <- readTVar b
  writeTVar b (xs ++ [x])

get :: Buf a -> STM (a)
get b = do
  xs <- readTVar b
  case xs of
    []      -> retry
    (x:xs)  -> do
      writeTVar b xs
      return x
```

# Productores y Consumidores sobre el Buffer

```
writer :: Buf Int -> IO ()
writer b = wloop 1000
  where
    wloop 0 = return ()
    wloop n = do atomically (put b n)
                  wloop (n-1)

reader :: Buf Int -> IO ()
reader b = rloop
  where
    rloop = do v <- atomically (get b)
                putStrLn ("Consumo " ++ show v)
                rloop

main :: IO ()
main = do
  b <- atomically newBuf
  writer b
  t <- async (reader b)
  wait t
```

# Algunas clases de STM

| Estructura | Capacidad       | esperan                      |
|------------|-----------------|------------------------------|
| TVar       | Ninguno (Celda) | No                           |
| TQueue     | Ilimitada       | readTQueue (vacía)           |
| TBQueue    | Acotada         | read (vacía) / write (llena) |
| TMVar      | Exactamente 1   | take (vacía) / put (llena)   |
| TChan      | Ilimitada       | readTChan (vacía)            |
| TArray     | Fija Indexada   | No                           |

# El Problema del Barbero Durmiente

- **El Barbero:** Ejecuta un bucle infinito de atención. Si la sala está vacía, se sienta en su silla y se duerme profundamente.
- **La Sala de Espera:** Cuenta con un número estrictamente limitado de  $N$  asientos para los clientes.
- **Los Clientes:** Al llegar, inspeccionan la barbería. Si el barbero duerme, lo despiertan para el corte. Si está ocupado pero hay sillas, se sientan. Si no hay lugar, se retiran de inmediato.

# El Barbero en STM

```
import Control.Concurrent (forkIO, threadDelay)
import Control.Concurrent.Async (async, wait)
import Control.Concurrent.STM

data Barberia = Barberia {
    sillasLibres :: TVar Int,
    capacidadMax :: Int,
    colaEspera   :: TQueue String
}

nuevaBarberia :: Int -> STM Barberia
nuevaBarberia n = do
    libres <- newTVar n
    cola   <- newTQueue
    return (Barberia libres n cola)
```

# Lógica de Sincronización del Barbero y Clientes

```
entrar :: Barberia -> String -> STM Bool
entrar b cliente = do
    libres <- readTVar (sillasLibres b)
    if libres == 0
        then return False
        else do
            writeTVar (sillasLibres b) (libres - 1)
            writeTQueue (colaEspera b) cliente
            return True

cliente :: Barberia -> String -> IO ()
cliente b nombre = do
    sentado <- atomically (entrar b nombre)
    if sentado
        then putStrLn ("Cliente " ++ nombre ++ ": Sentado a esperar.")
        else putStrLn ("Cliente " ++ nombre ++ ": Barberia llena, me voy."
    )
```

# El Ciclo de Trabajo Concurrente

```
atender :: Barberia -> STM String
atender b = do
    cliente <- readTQueue (colaEspera b)
    libres  <- readTVar (sillasLibres b)
    writeTVar (sillasLibres b) (libres + 1)
    return cliente

barbero :: Barberia -> IO ()
barbero b = bloop
    where
        bloop = do
            cliente <- atomically (atender b)
            putStrLn ("Barbero: Cortando el pelo a " ++ cliente)
            threadDelay 1500000 -- Tiempo del corte de pelo
            bloop

main :: IO ()
main = do
    b <- atomically (nuevaBarberia 3) -- Barberia con 3 sillas de espera
    t <- async (barbero b) -- Arranca el barbero (puede empezar durmiendo)
    mapM (async . cliente b) ["Juan", "Ana", "Pedro", "Susana", "Fran"]
    wait t
```

# Fumadores de Cigarrillos

- **Tres Fumadores:** Cada uno posee un suministro infinito de un único ingrediente indispensable: Tabaco, Papel o Fósforos. Para fumar, un individuo necesita reunir los tres elementos de forma simultánea.
- **El Agente:** Coloca al azar **dos ingredientes diferentes** sobre la mesa compartida. El fumador al que le falte exactamente esa combinación debe tomarlos de inmediato, armar su cigarrillo y fumar.

# Problema de los Fumadores: Estructura de datos

```
data Mesa = Mesa {  
    tabaco    :: TVar Bool,  
    papel     :: TVar Bool,  
    fosforo   :: TVar Bool  
}
```

```
nuevaMesa :: STM Mesa  
nuevaMesa = do  
    t <- newTVar False  
    p <- newTVar False  
    f <- newTVar False  
    return (Mesa t p f)
```

# Problema de los Fumadores: Fumadores

```
esperarIngredientes :: Mesa -> TVar Bool -> TVar Bool -> STM ()
esperarIngredientes mesa ing1 ing2 = do
    disponibles1 <- readTVar ing1
    disponibles2 <- readTVar ing2

    -- Dormir si no estan
    check (disponibles1 && disponibles2)

    -- Limpiar mesa
    writeTVar ing1 False
    writeTVar ing2 False

fumador :: String -> Mesa -> (Mesa -> TVar Bool) -> (Mesa -> TVar Bool) -> IO ()
fumador n m ing1 ing2 = floop
    where
        floop = do
            atomically (esperarIngredientes m (ing1 m) (ing2 m))
            putStrLn ("Fumador " ++ n)
            threadDelay 1000000 -- Tiempo fumando
            floop
```

# Problema de los Fumadores: Agente

```
proveer :: Mesa -> (Mesa -> TVar Bool) -> (Mesa -> TVar Bool) -> STM ()
proveer mesa ing1 ing2 = do
    t <- readTVar (tabaco mesa)
    p <- readTVar (papel mesa)
    f <- readTVar (fosforo mesa)
    check (not t && not p && not f)
    writeTVar (ing1 mesa) True
    writeTVar (ing2 mesa) True

agente :: String -> Mesa -> (Mesa -> TVar Bool) -> (Mesa -> TVar Bool) ->
    IO ()
agente n m ing1 ing2 = aloop
    where
        aloop = do
            do
                atomically (proveer m ing1 ing2)
                putStrLn $ "\nAgente: " ++ n
                threadDelay 10000 -- Tiempo fumando
            aloop
```

# Problema de los Fumadores: Principal

```
main :: IO ()
main = do
  m <- atomically nuevaMesa
  async (fumador "tabaco y papel" m tabaco papel)
  async (fumador "papel y fosforo" m papel fosforo)
  async (fumador "tabaco y fosforo" m tabaco fosforo)
  async (agente "tabaco y papel" m tabaco papel)
  async (agente "papel y fosforo" m papel fosforo)
  t <- async (agente "tabaco y fosforo" m tabaco fosforo)
  wait t
```